Python Assignment
Topic-6: Generators and Iterators

Q1. Understanding how generators work in Python.

Ans.

Generators are special functions that produce values one by one instead of returning all values at once. They use the yield keyword which pauses the function and returns a value. When we call the generator again, it continues from where it stopped. Generators help in saving memory because they do not store all values at once. They are very useful when working with large data. They make programs faster and more efficient. Generators are easy to use with loops and are an important concept in Python.

Example:

```
def numbers():
    yield 1
    yield 2
    yield 3

for n in numbers():
    print(n, "- Anshu")
```

Q2. Difference between yield and return.

Ans.

The return statement is used to send a value and stop the function completely. Once return is executed, the function ends. The yield statement is used in generators and it pauses the function instead of ending it. It allows the function to continue from the same place next time. Return gives only one value, but yield can give multiple values one by one. Yield is useful for saving memory and handling large data. It is mainly used in generator functions.

Example:

```
def demo():
    yield 5
    yield 10
```

```
for i in demo():
    print(i)
```

Q3. Understanding iterators and creating custom iterators.

Ans.

An iterator is an object that allows us to access elements one at a time. Lists, tuples, and strings are iterable objects in Python. We can create an iterator using the iter() function and get values using next(). Custom iterators can be created using classes with special methods like __iter__() and __next__(). Iterators help in processing data step by step. They are useful when working with loops and large collections of data. This concept helps in efficient data handling.

Example:

```
data = [10,20,30]
it = iter(data)
print(next(it))
print(next(it))
```